Computational Economics

Lab: Working with AI Agents

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Lab Overview

- **Choose and direct an agent**
  - Model families, evaluation criteria, and agent applications
  - Tool execution, CLI, and MCP
  - A concrete task prompt
- **Manage the information it uses**
  - Context, tokens, compaction, and caching
  - Project instructions and reusable skills
- **Check and reuse its work**
  - Inspect the result and rerun the program
  - Practice with a reusable script

LLM Families and Developers

A **large language model (LLM)** processes context and generates text, code, or requests to perform actions.

- **GPT family** — developed by **OpenAI**.
  Examples: GPT-6 Astra, GPT-6.1 Sol, GPT-6 Luna.
- **Claude family** — developed by **Anthropic**.
  Examples: Claude Fable 5, Claude Opus 5.5, Claude Sonnet 5.5.
- **Gemini family** — developed by **Google DeepMind**.
  Examples: Gemini 4 Argon, Gemini 3.8 Flash.
- **Grok family** — developed by **xAI**.
  Examples: Grok 4.20, Grok 4.7.
- **DeepSeek family** — developed by **DeepSeek**.
  Examples: DeepSeek-V4-Pro, DeepSeek-V4.1-Flash.

How LLMs Are Evaluated

LLMs are commonly judged along the following dimensions.

- **Task performance**: accuracy or success on one attempt (**pass@1**). For example, Terminal-Bench tests tasks performed with terminal tools.
- **Reliability**: factual errors, instruction-following failures, and consistency across repeated runs.
- **Token use and cost**: a **token** is a unit of text, often part of a word. Cost depends on token usage (including reported reasoning) and token prices; a useful measure is **cost per completed task**.
- **Speed**: time to the first answer token, output tokens per second, and total completion time. Longer reasoning can increase total time.
- **Input capabilities**: supported languages and **multimodal inputs**, such as text, images, audio, and video.
- **Long-context performance**: context-window size and accuracy when using long documents.

Scores depend on the test setup: extra attempts, additional tools, or more reasoning time can improve results while increasing cost.

Reading: Artificial Analysis, Benchmarking methodology.

The Harness: Tools and Interfaces

A **harness** is the software surrounding an LLM. It supplies tools, manages context and permissions, and returns tool results to the LLM.

- **Files and coding:** search, read, create, and edit files.
- **Execution:** run Python, terminal commands, and tests.
- **Web access:** search the web, retrieve pages, and operate a browser.
- **External services:** access a database, cloud drive, or other connected service.

Tools can be accessed through different interfaces:

- A **command-line interface (CLI)** operates software through text commands. The harness can run `git status` to inspect repository changes.
- **Model Context Protocol (MCP)** standardizes how applications connect to servers providing tools and information.

How the Harness Accesses Tools

The harness executes requests and supplies results as input to the next LLM call.

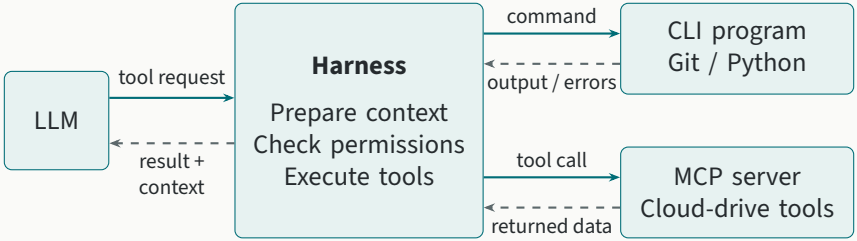

Figure 1: Solid arrows: requests. Dashed arrows: returned information.

- The **command runner** and **MCP client** are components used by the harness.
- CLI and MCP can work together in one task; an MCP server can itself run a CLI program. These are examples of tool access, not an exhaustive list.

The Agent: Coordinating a Task

LLM + harness = AI agent

Task: turn a CSV in a shared drive into a checked chart.

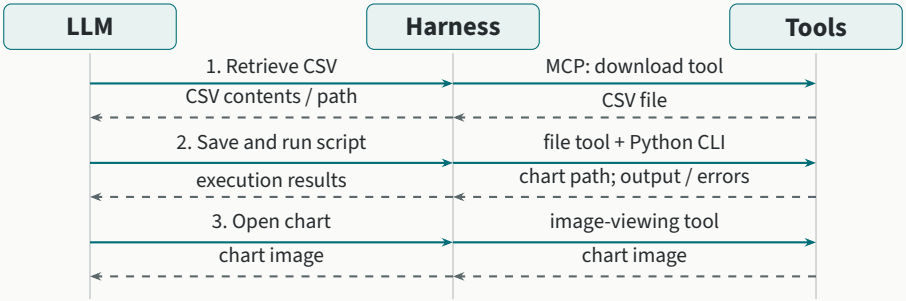

Figure 2: Download, run, and inspect; revise if needed, then report.

Agent applications: Codex (OpenAI), Claude Code (Anthropic), Cursor Agent (Cursor), Gemini CLI (Google), and GitHub Copilot (GitHub).

From a Vague Request to a Concrete Task

A concrete prompt makes the intended result and the criteria for success explicit.

Suppose you receive a new temperature file each week.

Each `data.csv` contains `date` and `temperature_c`. You want a chart now and a procedure you can reuse next week.

Vague prompt

“Make a chart from my data.”

Concrete prompt

“Write `plot_temperature.py` to read `data.csv`, sort by date, and save a line chart to `results/temperature.png`. Label the axes Date and Temperature (Celsius).

Keep the input unchanged. Use the project Python environment. Run the script, check the dates, values, and units against the CSV, and inspect the chart. Report the files changed and the command used.”

Reuse the checked script next week to plot the new file.

Reading: OpenAI, Prompt engineering; CarpeDiem, Programs as AI tools.

The Prompt Is Part of the Context

Context is the input the harness supplies to the model for its next response.

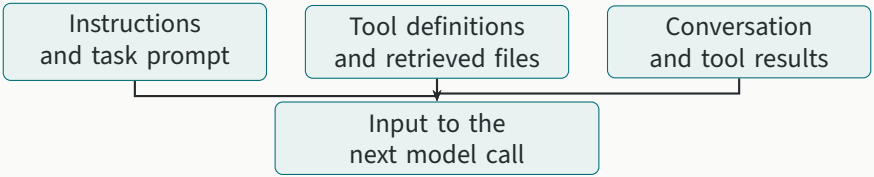

Figure 3: The harness assembles the model input.

- The **context window** limits the number of tokens a model call can accommodate.
- The harness loads relevant files and selects or summarizes earlier history.

“Fix the axis label” can accompany the CSV, script, and history: a short message can have a large input.

Keep the Working Context Relevant

Context engineering selects and maintains relevant information. Even a large context can contain details the model overlooks.

- **Load what helps.** For a plotting error, retrieve the relevant code and error message.
- **Keep related work together.** Revise the chart in one session; start a new one for unrelated work.
- **Preserve decisions. Compaction** replaces earlier detail with a shorter summary. Check that essential requirements survive.
- **Separate lengthy side tasks.** A **subagent** can investigate in a separate context and return a concise result.

A handoff to a fresh session

"The plotting script runs. Keep dates sorted and use Celsius. Next: compare the chart with `data.csv`."

Saved notes or **memory** help later sessions only when retrieved into their context.

Reading: CarpeDiem, Agent cost-saving tips (Part 1).

Context Caching Reuses Computation

Context caching reuses computation for an unchanged input prefix when a compatible cache is available.

Figure 4: Reuse A + B; process the new C.

- Eligible cached input can cost less to process. Retention and prices depend on the provider.
- Changing earlier instructions or history can invalidate reuse. Cached text still occupies context space.
- Cache expiry requires recomputation; compaction changes which information the model receives.

Judge the completed task: a high cache-hit rate alone does not imply low cost or good results.

Reading: CarpeDiem, Agent cost-saving tips (Part 2).

Reuse Instructions and Procedures

Keep recurring guidance in files the agent can load.

Project instructions
Rules for work in this project. Codex uses `AGENTS.md`; filenames and scope vary across applications.

Chart project rules

- Keep input data unchanged.
- Include units on axes.
- Save figures in `results/`.

A **skill**
A reusable procedure, with optional code and references. In the Agent Skills format:

```
plot-data/
  SKILL.md
  scripts/
  references/
```

Plotting procedure: read columns and units, create the figure, inspect it, and report checks.

The agent discovers a skill by its name and description, then loads its instructions when relevant. The skill guides tool use.

Reading: Agent Skills, Overview.

Verify the Work Before Reusing It

A successful tool call or a confident completion message is evidence to inspect; compare the actual result with the task.

1. **Inspect the changes.** Review the script and confirm the input file was preserved.
2. **Reproduce the run.** Record the command and Python environment; rerun the script.
3. **Check the output.** Compare dates and values with the CSV; inspect axis labels and units.
4. **Keep the checked version.** Commit the script with Git so it can be reused or revised.

For temperatures of 20, 22, and 21 Celsius, the chart should preserve the date order and have its highest point at 22.

A **sandbox** restricts execution; **approvals** authorize actions. Retrieved files may contain **prompt injection**: instructions that try to redirect the agent. Treat external content as data to assess.

Practice: Run, Check, and Reuse

Create a working folder and save this practice file as `data.csv`:

```
date,temperature_c
2026-10-01,20
2026-10-02,22
2026-10-03,21
```

1. Choose an agent application and a supported model. Give it the chart prompt and access to this folder.
2. Follow one tool call: inspect what it reads or runs and the result returned to the model.
3. Review the script, rerun it, and check the chart against the CSV.
4. Add one observation to the practice CSV yourself; rerun the same script.

Extend the workflow: save a project rule for axis units, or package the plotting procedure as a skill when supported.

References and Further Reading

1. **OpenAI, Agent and Codex documentation. [Documentation]**
   - Codex as a platform: build on the open agent harness
   - Prompt engineering
   - Prompt caching
2. **Anthropic, Claude model and Claude Code documentation. [Documentation]**
   - Claude Code: Overview
3. **Microsoft, Visual Studio Code: agent and terminal documentation. [Documentation]**
   - Understand agent harnesses
4. **Model Context Protocol contributors, Model Context Protocol documentation. [Documentation]**
   - Architecture overview

References and Further Reading

5. **Anthropic (2025), Effective context engineering for AI agents. [Technical article]**
   - Context engineering vs. prompt engineering
6. **Agent Skills contributors, Agent Skills documentation. [Documentation]**
   - Agent Skills Overview
7. **CarpeDiem (2026), AI Learning from Zero: practical agent tutorials. [Video series]**
   - Code is the most handy tool for AI
   - Agent cost-saving tips (Part 1): tokens and context
   - Agent cost-saving tips (Part 2): context caching
8. **Artificial Analysis, Benchmarking methodology. [Web resource]**